Stephanie Gillihan
CSPB 4502 Data Mining
Project Part 4: Final Report

Chess Insights: Result Predictors and Play Sequences by Player Ranking

Abstract
Chess Insights investigates which characteristics of online chess games are most strongly associated with White wins, Black wins, and draws, whether move-sequence patterns differ across player rating levels, and how prediction changes as more of a game is observed. The analysis used a random sample of 5,000,000 games from the February 2026 Lichess standard-rated dataset, with 4,980,959 games retained after cleaning. Methods included descriptive statistics, chi-squared tests, frequent move-sequence analysis using support, confidence, and lift, and Logistic Regression and Random Forest classification models. Results showed that relative player rating was the strongest pre-game predictor of outcome, accounting for most Random Forest feature importance, while move-sequence associations were weaker and varied by rating level. Draw-associated sequences showed the strongest pattern effects, but the sample size for draws was small compared to wins and losses. Checkpoint-based outcome prediction improved as more moves were observed but did not outperform the pre-game model until a substantial portion of the game had already been played. Overall, the results indicate that player skill is the dominant predictor of chess outcomes, while move patterns provide more limited, rating-dependent, and context-specific signals.

Introduction

Large datasets from online chess platforms such as Lichess make chess match data available at an unprecedented scale. Game records from these sources contain detailed information about player ratings, game results, openings, time controls, and complete move sequences, offering the opportunity to mine for useful patterns and investigate outcome predictors.

The *Chess Insights* project investigates three primary questions:

1. Which game and player characteristics are most strongly associated with White wins, Black wins, and draws?
2. Which opening or move-sequence patterns are associated with White wins, Black wins, and draws?
3. Do any relationships identified in the course of investigating the first two questions differ across player rating levels?

These questions are important because common chess advice often treats openings and move patterns as broadly applicable, even though their effectiveness may depend on the skill level of the players involved.

To address these questions, this project analyzes a large sample of standard-rated games from the February 2026 Lichess database. The analysis combines descriptive statistics,

categorical comparisons, frequent move-sequence analysis, and classification models. Emphasis is placed on comparing outcome patterns across rating bins and distinguishing between information available before a game begins and information that becomes available as the game progresses.

In addition to identifying patterns in chess outcomes, this project addresses practical challenges involving data parsing, feature engineering, memory management, sampling, and scalable analysis for a very large dataset.

Related Work

Previous chess data studies show that online chess platforms can support large-scale analysis of player behavior, game outcomes, and opening choices. De Marzo and Servedio used Lichess data to study chess openings as a relatedness network, measuring similarity between openings and using online community data to understand the structure and complexity of opening theory. Their work is relevant because it treats openings not only as isolated labels, but as patterns that can be compared and grouped based on how they are played.

Chowdhary et al. used large-scale Lichess data to quantify human performance in chess. Their study notes that the Lichess dataset includes detailed information about played games, including moves, openings, win/loss outcomes, and player skill level. This supports the feasibility of my project because the same general data source contains the features needed to connect player rating, move behavior, and game result.

Recent machine learning work has also explored prediction of chess game outcomes. Samara developed and compared machine learning models for classifying chess game results as White win, Black win, or draw using Lichess data, player rating information, and game metadata. This is closely related to my predictive modeling goal, but my project places more emphasis on interpretable aggregate comparisons, opening and move-sequence patterns, and differences across rating bins rather than only model performance.

Chess Insights differs from previous studies in its scope and emphasis. Rather than focusing only on opening similarity, only human performance, or only predictive accuracy, this project combines interpretable descriptive analysis, frequent sequence analysis, and classification. It also explicitly compares results across player rating bins, which should help determine whether predictors of success differ between lower-rated, intermediate, and higher-rated players.

Dataset

The dataset is the Lichess dataset from February 2026. The file was 27.8GB compressed, and 184.12GB decompressed. It contains information for 84,600,043 chess games. The dataset citation is provided in the References section. This dataset contains the key fields needed for the project: game results, player ratings, game metadata, and full move sequences. Its large size also makes it appropriate for a data mining project because it

supports analysis across rating groups and less common opening patterns while requiring careful preprocessing and computational planning. The ethical risk associated with this dataset is low because the analysis uses publicly released game records and focuses on aggregate gameplay patterns rather than identifying or profiling individual players.

Some basic knowledge of chess terminology is needed to work with this dataset or to understand the results of this project. Please refer to Figure 1: Chess Terminology Primer. At a high level, the Lichess dataset is a plain-text chess game format known as Portable Game Notation (PGN). Each game is a header block of bracketed tag-value pairs followed by a movetext block. There is a specialized library within Python called python-chess that handles parsing and converting the movetext into Standard Algebraic Notation (SAN).

The data frame after parsing contains fourteen columns: game_id, event, result, utc_date, white_elo, black_elo, white_rating_diff, black_rating_diff, eco, opening, time_control, termination, num_plies, and moves. Missing values, incomplete games, and other data quality issues were addressed in the data cleaning section. Please see the list below for column names and definitions:

Game ID – game_id – integer column containing a unique game ID
Event – event – a string description of the type of game
Result – result – string column indicating game outcome in the format 1-0 or 0-1
UTC Date – utc_date – a string column indicating the game date in format YYYY.MM.DD
White ELO – white_elo – a string column representing white player score
Black ELO – black_elo – string column representing black player score
White Rating Differential – white_rating_diff – string column representing the white player advantage based on player rating
Black Rating Differential – black_rating_diff – string column representing the black player advantage based on player rating
ECO Code – eco – string column representing the code for the opening move in format LetterNumber (ex: B12)
Opening – string column containing a description of the opening move used
Time Control – string column containing the type of time control in format ##+#
Termination – termination – string column indicating the type of game termination; Normal,
Number of Plies – num_plies – integer column containing the total number of plies in the game
Moves – moves – an object column containing a list of all moves in the game

Techniques Applied

Languages, Libraries, and Computational Considerations

Python was the primary programming language used for *Chess Insights* because it supports the full workflow from parsing to modeling and visualization. Several Python libraries were used to support various processing and analysis work. Zstandard, python-chess, pandas, NumPy, and PyArrow are used for parsing, extraction, cleaning, transformation, sampling, aggregation, and storage.

For modeling and evaluation, scikit-learn supported train/test splits, classification models, confusion matrices, and performance metrics. For visualization, matplotlib and seaborn were used to create charts and tables that support the final report.

The tools above worked very well on an initial 50k game sample dataset but did not automatically or easily scale to the full 85 million game dataset. It was necessary to add explicit deletion and garbage collection, as well as chunked processing and sampling for memory management. I also refactored the code to use multiple computing cores. Despite the efforts to make the data analysis pipeline as computationally efficient as possible, I still could not process 85 million games on my PC. The initial plan was to run the data pipeline notebook on the full 85 million game dataset using rented cloud hardware. However, there were two major blockers that prevented successful execution of that plan. First, after registering for cloud computing services, I discovered that it is standard practice for those services to limit access and/or computational resources for new accounts. There was an option to request an exemption, but it was uncertain how long it might take to get an exemption. Secondly, running the full 85 million game data set was estimated to take ~120 GB of peak RAM, which is fairly significant in cost. I determined that it was unnecessary to spend more than $50 to have the satisfaction of running the full dataset.

Instead, I modified the code to accommodate random sampling of games using random byte positions. The sampler jumps to a random byte position in the dataset and selects the first game that begins after that byte position. This strategy eliminated the need to parse the entire 85 million games in order to select a subset of truly random games. While the random-byte-sampling is not actually fully random, it is close enough. There is a small bias towards selecting games that immediately follow a particularly long game. However, I predict that very little difference would be found between results for the entire dataset and results for the subsample used for this project.

I first attempted analysis on a random-byte subsample of 15 million games, but that attempt failed due to memory issues. After recalculating the maximum number of games my PC could handle, I successfully ran the data pipeline notebook on a 5 million games subset. If I were extending the project, I could repeat the random 5 million game analysis ten times and compare the results across ten different 5 million game subsamples. Please see Figure 2: Data Ingestion and Parsing.

This experience working with Python on a massive dataset demonstrated that while it excels at tabular manipulation on fairly large datasets, there is a ceiling on what standard libraries can handle when performing parsing and modeling. I learned a lot of memory management techniques and computational efficiency techniques in the course of this project.

Data Cleaning and Feature Engineering

The data was already relatively clean, and only 19,041 out of 5,000,000 games were removed in course of data cleaning. 10,004 games were removed for missing values, and most of the remaining games were removed for lacking the minimum game length of two plies, or for implausible Elo ratings. Please see Figure 3: Before and after data cleaning for summaries of before cleaning and after cleaning numbers for key columns.

The following features were added to support analysis: rating_diff, avg_rating, rating_bin, num_moves, tc_cat, eco_family, opening_family, white_wins, higher_rated_wins. Please see the list below for a definition of each engineered feature column:

Rating Difference – rating_diff – positive values mean white player is rated higher; (white_elo - black_elo)
Average Rating – avg_rating – serves as a proxy for game quality level; (white_elo + black_elo)/2
Rating Bin – Rating_bin – Groups average game rating into groups: <1000, 1000-1499, 1500-1999, and >=2000
Number of Moves – num_moves – the count of full moves; num_plies//2
Time Control Category – tc_cat – Type of time control derived from the time control column; Bullet, Blitz, Rapid, or Classical
ECO Family – eco_family – the first letter of the broad opening family (ECO codes A-E)
Opening Family – opening_family – the text before the first colon in the Opening tag
White Wins – white_wins – a binary representation of game outcome; 1 indicates white wins and 0 indicates black win or a draw
Higher Rated Wins – higher_rated_wins – a binary representation of game outcome; 1 indicates the higher ranked player wins and 0 indicates a draw or the lower ranked player wins

Descriptive Statistics and Categorical Comparisons

Descriptive statistics such as counts, percentages, means, medians, standard deviations, quartiles, and interquartile ranges are reported for player ratings, rating differences, game length, outcome frequencies, and opening frequencies. These statistics help to quantify and understand the general makeup of the dataset.

The mean and median player ratings were 1642 and 1654 for both black and white players. The average game rating mirrored the individual player ratings. The average and median rating difference between players were both near 0. The average number of complete moves per game was 33, and the median was 31. 6% of games had an average rating <1000, 29.6% of games had an average rating 1000-1499, 45.7% of games had an average rating 1500-1999, and 18.7% of games had an average rating >=2000. Please see Figure 4: Summary Statistics of Selected Features and Figure 5: Engineered Feature Summaries for a statistical summary of these features.

The distributions of player rating distribution, rating difference distribution, and average game rating were all bell curved, as shown in Figure 7: Player Rating Analysis and Rating Difference Analysis.

Overall outcome distribution was near 50/50, with white player winning 49.7% of the games, black player winning 46.4% of the games, and 3.9% of the games resulting in a draw. Game outcomes were also summarized by rating bins, with similar distributions across all rating bins. Please see Figure 6: Outcome Distribution.

For categorical comparisons, I used chi-squared tests on contingency tables for features including opening category by outcome, rating bin by outcome, time control by outcome, and white first move by outcome. A chi-squared test checks whether two categorical variables are associated by comparing observed counts with the counts expected if the variables were independent. A small p-value suggests the observed relationship is unlikely to be due to random chance. All of the chi-squared tests had small p-values, but the outcome breakdown rarely deviated from roughly 50/50. Please see Figure 9: Chi Square Tests for a summary of chi-squared tests.

Move Sequence Patterns and Classification Model Training

Move-sequence analysis was used to determine whether recurring patterns of play had strong association with White wins, Black wins, or draws, and whether those associations differed across player rating levels. The preliminary approach to move sequence analysis represented moves according to both the move itself and its exact position within the game. However, this method was too restrictive because similar patterns could not be identified when they occur at different points in a game. I also experimented with analyzing White-only and Black-only sequences separately before determining that this removed important information about the interaction between the two players. The final approach searched for contiguous sequences of 6, 8, and 10 plies across the complete move history, preserving alternating moves of White and Black players. Sequences identified as candidates were evaluated separately for White wins, Black wins, and draws using support, confidence, and lift. Support measures how frequently a sequence occurs in the dataset, confidence measures the proportion of games containing the sequence that results in a particular outcome, and lift compares that outcome rate with its overall baseline probability. A lift greater than 1 indicates that the outcome occurs more frequently among games containing the sequence than it does in the dataset overall. Minimum percentage-based support thresholds were used to exclude rare sequences whose apparent associations could be driven by very small sample sizes. Sequence results were also calculated within the four rating bins to determine whether the same patterns were associated with outcomes across different skill levels. Please refer to Figure 10: Top White and Black win Move Sequences, Figure 11: Top Combined Move Sequences by Outcome, and Figure 12: Move Sequences by Player Rating.

Two classification approaches were then used to investigate game-outcome prediction. Model A was a pre-game model, designed to use only information that would be known

before play begins (player rating, player color, time control category). Logistic Regression provided an interpretable linear baseline, while Random Forest was used to determine whether nonlinear relationships or interactions among the same features improved predictive performance. Model C was designed to be a live-game outcome predictor, using a checkpoint approach to evaluate whether prediction improves as more of the game becomes observable. Games were evaluated at equal interval ply checkpoints, and mined move sequence pattern presence at each checkpoint was used to update the predicted outcome. This approach allowed prediction quality to be measured as a function of game progression rather than treating information from the completed game as if it had been available beforehand. Because the complete checkpoint dataset would have been too large to hold in memory, training observations were maintained using a bounded reservoir while test observations were accumulated and evaluated in chunks.

Classification model performance was evaluated using accuracy, precision, recall, F1 score, ROC-AUC, and confusion matrices. Accuracy measures the proportion of all predictions that are correct. Precision measures how frequently positive predictions are correct, while recall measures how frequently actual positive cases are successfully identified. F1 score is the harmonic mean of precision and recall and provides a combined measure when both types of performance are important. ROC-AUC evaluates how effectively a binary classifier separates the two classes across possible classification thresholds. Confusion matrices were also examined to identify whether models systematically favored or failed to predict particular outcome classes. These complementary metrics were especially important because draws represented a relatively small proportion of the dataset, meaning that accuracy alone could obscure poor performance on the minority class.

Key Results

Player Rating as a Predictor of Outcome

Player rating was the strongest pre-game predictor identified in the project. In the pre-game Random Forest model, the player's rating advantage, represented by own_rating_diff, accounted for approximately 91.2% of total feature importance. Average player rating contributed another 5.0%, meaning the two rating-related variables together represented approximately 96% of the model's total feature importance. Other variables, including player color and time-control category, contributed comparatively little.

This finding was also reflected in the predictive performance of the models. The Logistic Regression pre-game model achieved 55.86% accuracy and a ROC-AUC of 0.5867, while the Random Forest achieved 55.88% accuracy and a ROC-AUC of 0.5872. These results were only slightly higher than the simple heuristic of predicting that the higher-rated player would win, which achieved approximately 55.29% accuracy. The similarity between these results suggests that much of the available pre-game predictive information is already captured by relative player rating.

Player color still showed a small independent effect. In the Logistic Regression model, playing White had a positive coefficient and playing Black had a negative coefficient after accounting for rating-related variables. The estimated difference corresponded to an approximately 0.13–0.14 change in log-odds between White and Black, which was very similar to the effect observed in the smaller development sample. This suggests that White's first-move advantage is measurable but substantially smaller than the effect of relative player skill.

Move Sequence Associations with Game Outcomes

Frequent contiguous move sequences of 6, 8, and 10 plies were analyzed using support, confidence, and lift. A fixed percentage-based minimum support threshold was used so that only sufficiently common patterns were retained. In the final sample, 83 sequences met the support threshold for White wins, 83 for Black wins, and 59 for draws. The strongest White-win sequence had a lift of 1.124, while the strongest Black-win sequence had a lift of 1.144. These values indicate only modest increases above the baseline probability of the corresponding outcome. Both values were lower than the strongest lifts observed in the 50,000-game sample, suggesting that some of the stronger apparent win-associated patterns in the smaller sample were partly the result of sampling variability.

Draws produced a different pattern. The strongest draw-associated sequence was the sequence e4 e5 Nf3 Nc6 Bb5 Nf6 (known as the Ruy Lopez/Berlin Defense), which had a lift of 2.069. Several other highly ranked draw-associated sequences were also related to the Ruy Lopez/Berlin Defense. Unlike the White-win and Black-win patterns, the strongest draw association became larger in the full analysis than in the preliminary sample. This suggests that some opening and early-game sequences are much more distinctive for draws than for decisive outcomes.

Move Sequences Across Rating Levels

The move-sequence analysis also showed that associations differed substantially across rating levels. No single sequence was the strongest pattern in more than one rating bin. This supports the hypothesis that move-pattern effects are rating-dependent rather than universal across all players.

The strongest White-win sequence became progressively less distinctive as player rating increased. Its lift declined from 1.203 in games with average rating below 1000, to 1.083 for ratings 1000–1499, 1.042 for 1500–1999, and 1.003 for ratings of 2000 or higher. At the highest rating level, this represents almost no difference from the baseline White-win probability. In addition, the strongest overall sequence in the two highest rating bins was associated with Black wins rather than White wins. These results suggest that specific move sequences may provide a larger advantage or reflect larger differences in play at lower ratings, while stronger players are less likely to exhibit simple outcome-associated patterns.

Outcome Prediction as Game is Observed

The checkpoint model was designed to evaluate whether prediction improves as additional moves become available. Random Forest accuracy increased steadily as more of the game was observed, rising from 53.98% at ply 10 to 61.47% at ply 120 and approximately 74.4% at ply 200. However, this improvement requires important context. The checkpoint model did not exceed the pre-game Random Forest accuracy of 55.88% until approximately ply 75. Because the median game length in the dataset was approximately 63 plies, the checkpoint model typically needed to observe more than half of a game before outperforming a simple pre-game model based primarily on player ratings. This limitation indicates that the checkpoint model may not be very useful as an early-game predictor.

The apparent improvement in accuracy at very late checkpoints should also be interpreted cautiously because the number of games remaining in the sample decreases substantially as checkpoint ply increases. For example, approximately 1.97 million observations were available at ply 10, compared with only about 1,580 at ply 200. Late-game accuracy estimates are therefore based on a much smaller and less representative subset of unusually long games.

An additional complication was that ROC-AUC generally declined as checkpoint accuracy increased, falling from approximately 0.581 at ply 10 to 0.534 at ply 200. This indicates that higher late-game accuracy did not necessarily correspond to better overall ranking of positive and negative cases. Instead, later game states may simply contain more games in which the eventual result has become easier to classify with a hard decision threshold.

Comparison of Classification Models

Across the pre-game models, Logistic Regression and Random Forest performed very similarly. Random Forest achieved 55.88% accuracy compared with 55.86% for Logistic Regression, and the ROC-AUC values were also nearly identical. This suggests that the additional nonlinear flexibility of Random Forest provided little benefit when only pre-game rating and metadata features were available.

The pooled checkpoint models produced a similar result. Logistic Regression achieved slightly higher pooled accuracy than Random Forest, while Random Forest produced a slightly higher ROC-AUC. However, the checkpoint Random Forest displayed an important weakness: precision was approximately 0.697 while recall was only 0.065. This means that the model rarely predicted the positive class but was often correct when it did. The imbalance between precision and recall suggests a threshold, calibration, or class-imbalance problem and indicates that overall accuracy alone does not fully describe model quality.

The multiclass models also performed poorly when predicting draws. In the pre-game model, only 943 observations were predicted as draws across a test set containing more than 194,000 actual draws. The checkpoint model showed a similar tendency to avoid the draw class. This result is notable because frequent-pattern analysis identified strong draw-

associated move sequences, yet the rating-dominated classification models did not make effective use of that signal.

Overall Results

Taken together, the results indicate that relative player rating is substantially more useful for predicting chess outcomes than any individual opening, move sequence, or basic game metadata feature. Move sequences still contain meaningful information, but their associations with wins are generally modest and vary across player rating levels. Draw-associated sequences show a stronger and more consistent signal than sequences associated with White or Black wins.

The checkpoint analysis further demonstrates that outcome prediction improves as more of the game is observed, but meaningful improvement over a simple pre-game rating model occurs only after a substantial portion of the game has already been played. Overall, the findings support the conclusion that player skill remains the dominant predictor of game outcome, while move-sequence patterns provide more limited and context-dependent predictive information.

Applications

The findings from *Chess Insights* have several potential applications for chess training, game analysis, and future research. One practical use is in rating-specific player development. The move-sequence analysis showed that patterns associated with game outcomes differ across rating groups and that no single sequence was the strongest across multiple rating bins. Players and coaches could use rating-specific statistics to identify openings and early-game sequences that are most relevant to players at a particular level.

The results could also support chess analytics and prediction tools, especially on larger or more specialized chess datasets. The analysis could be repeated across multiple months of Lichess data to determine whether the identified relationships remain stable over time. Repeated random samples could also be compared to quantify the sampling variability introduced by the five-million-game subset used in this project. Future work could further separate games by time control, investigate narrower rating ranges, or use board-position features rather than raw move sequences. More advanced models could incorporate material balance, piece activity, king safety, or engine evaluations at specific checkpoints, which may capture game state more directly than move-sequence indicators alone.

Overall, the results can help players understand how skill level, color, and commonly played sequences relate to outcomes, but they do not provide a formula for selecting moves that will guarantee success. The relatively modest predictive performance of the classification models reinforces this limitation. Chess outcomes depend on the interaction of player skill and a large sequence of decisions, making simple statistical patterns useful for identifying tendencies but insufficient for determining the outcome of an individual game.

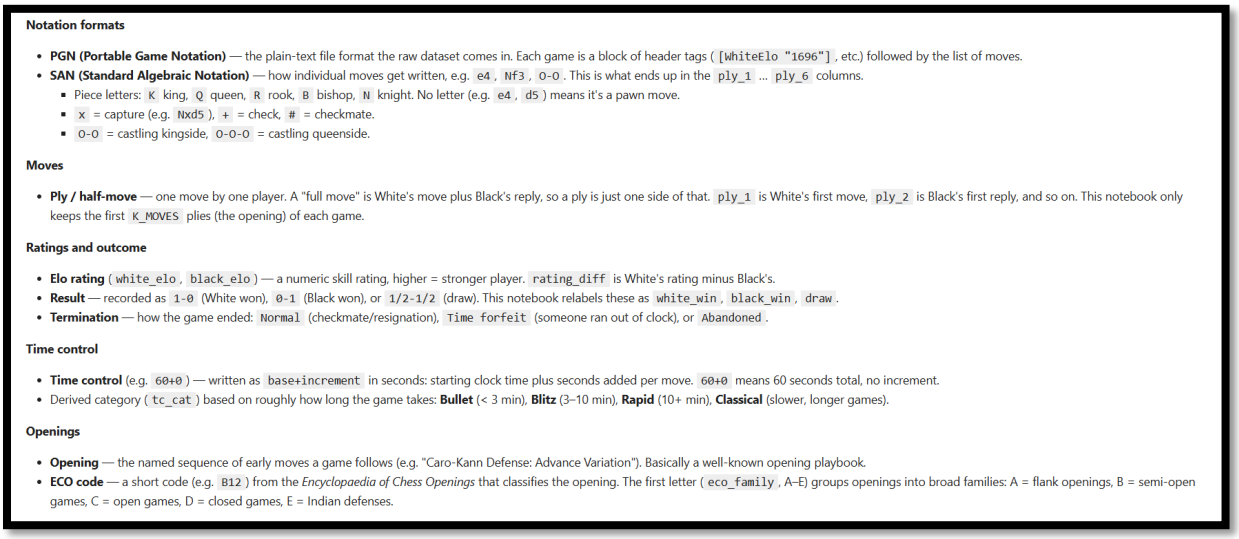
Conclusions and Limitations

The *Chess Insights* project examined which features of online chess games are most strongly associated with wins, losses, and draws, how those relationships vary across player rating levels, and whether prediction improves as more of a game is observed. Overall, the results show that relative player rating is the strongest and most consistent predictor of game outcome. Move-sequence patterns also contain useful information, but their effects are generally smaller, vary by rating level, and are strongest for draws rather than decisive outcomes. The checkpoint analysis further showed that prediction improves as more moves are observed, although the model does not clearly outperform a pre-game rating-based model until a substantial portion of the game has already been played.

Several limitations affect how these findings should be interpreted. The final analysis used a random sample of 5 million games rather than the full 84.6-million-game dataset because of computational and memory constraints. Although the sampling approach distributed selections throughout the source file, it may introduce a small amount of selection bias. Draws were also relatively rare, representing only 3.9% of games, which made them difficult for the classification models to predict despite the strong draw-associated patterns identified through sequence mining. Finally, the observed relationships are associative rather than causal; a move sequence with elevated lift should not be interpreted as directly causing a particular outcome.

Despite these limitations, the project successfully demonstrated that large-scale chess data can reveal meaningful differences in how player skill, game context, and move patterns relate to outcomes. The results suggest that chess success cannot be reduced to a single opening or winning sequence. Instead, player skill remains the dominant factor, while move patterns provide more limited and context-dependent signals that become increasingly informative as a game develops.

Visualization of Results



Notation formats

- **PGN (Portable Game Notation)** — the plain-text file format the raw dataset comes in. Each game is a block of header tags (`[WhiteElo "1696"]` , etc.) followed by the list of moves.
- **SAN (Standard Algebraic Notation)** — how individual moves get written, e.g. `e4` , `Nf3` , `O-O` . This is what ends up in the `ply_1` … `ply_6` columns.
  - Piece letters: `K` king, `Q` queen, `R` rook, `B` bishop, `N` knight. No letter (e.g. `e4` , `d5`) means it's a pawn move.
  - `x` = capture (e.g. `Nxd5`), `+` = check, `#` = checkmate.
  - `O-O` = castling kingside, `O-O-O` = castling queenside.

Moves

- **Ply / half-move** — one move by one player. A "full move" is White's move plus Black's reply, so a ply is just one side of that. `ply_1` is White's first move, `ply_2` is Black's first reply, and so on. This notebook only keeps the first `K_MOVES` plies (the opening) of each game.

Ratings and outcome

- **Elo rating** (`white_elo` , `black_elo`) — a numeric skill rating, higher = stronger player. `rating_diff` is White's rating minus Black's.
- **Result** — recorded as `1-0` (White won), `0-1` (Black won), or `1/2-1/2` (draw). This notebook relabels these as `white_win` , `black_win` , `draw` .
- **Termination** — how the game ended: `Normal` (checkmate/resignation), `Time forfeit` (someone ran out of clock), or `Abandoned` .

Time control

- **Time control** (e.g. `60+0`) — written as `base+increment` in seconds: starting clock time plus seconds added per move. `60+0` means 60 seconds total, no increment.
- Derived category (`tc_cat`) based on roughly how long the game takes: **Bullet** (< 3 min), **Blitz** (3–10 min), **Rapid** (10+ min), **Classical** (slower, longer games).

Openings

- **Opening** — the named sequence of early moves a game follows (e.g. "Caro-Kann Defense: Advance Variation"). Basically a well-known opening playbook.
- **ECO code** — a short code (e.g. `B12`) from the *Encyclopaedia of Chess Openings* that classifies the opening. The first letter (`eco_family` , A–E) groups openings into broad families: A = flank openings, B = semi-open games, C = open games, D = closed games, E = Indian defenses.

Figure 1: Chess Terminology Primer

```
Parsing a random cross-section of 5,000,000 games from PGN, as a standalone subprocess
(8 parallel workers, scattered across the whole file by byte position -- not the first N
games in file order) ...

  ... still sampling (1.0 min elapsed)
  ... still sampling (2.0 min elapsed)
  ... still sampling (3.0 min elapsed)
  ... still sampling (4.0 min elapsed)
```

```
  ... still sampling (93.0 min elapsed)
  ... still sampling (93.1 min elapsed)
Randomly sampled 5,000,000 distinct games using 8 workers in 6429.2s (778 games/s)
Saved 5,000,000 games to c:\Users\User\OneDrive\Documents\CSPB4502\DataMiningProject\dat
a\interim\games_raw_random.parquet

Raw DataFrame shape: (5000000, 14)
```

Figure 2: Data ingestion and parsing

```
Missing values in key columns (before cleaning):
  white_elo              0  (0.0%)
  black_elo              0  (0.0%)
  eco               10,004  (0.2%)
  opening           10,004  (0.2%)
  time_control           0  (0.0%)
  termination            0  (0.0%)
```

```
Missing values in key columns (after cleaning):
  white_elo              0  (0.0%)
  black_elo              0  (0.0%)
  eco                    0  (0.0%)
  opening                0  (0.0%)
  time_control           0  (0.0%)
  termination            0  (0.0%)
```

```
Games before cleaning: 5,000,000
Games removed:            19,041  (0.38%)
Games after cleaning:  4,980,959
```

Figure 3: Before and after data cleaning

```
=== Dataset Overview ===
Total games:     4,980,959
Date range:      2026.02.01 to 2026.02.28

                      n    mean  median    std      Q1      Q3    IQR
white_elo     4980959.0  1641.9  1654.0  404.9  1362.0  1923.0  561.0
black_elo     4980959.0  1641.6  1654.0  405.0  1361.0  1922.0  561.0
rating_diff   4980959.0     0.2     0.0  120.9   -32.0    33.0   65.0
avg_rating    4980959.0  1641.7  1655.0  400.4  1364.5  1920.0  555.5
num_plies     4980959.0    66.6    63.0   30.8    45.0    84.0   39.0
num_moves     4980959.0    33.0    31.0   15.4    22.0    42.0   20.0
```

```
Rating bin counts and percentages:
  1000-1499      1,475,244  (29.6%)
  1500-1999      2,274,967  (45.7%)
  2000+            930,105  (18.7%)
  <1000            300,643  (6.0%)
```

Figure 4: Summary Statistics of Select Features

```
Rating bin counts:
rating_bin
1000-1499    1475244
1500-1999    2274967
2000+         930105
<1000         300643
```

```
Time control category counts:
tc_cat
blitz        2381551
bullet       1774229
rapid         797167
classical      24786
unknown         3226
```

```
ECO family distribution:
eco_family
C    1510135
B    1449531
A    1305597
D     648416
E      67280
```

```
higher_rated_wins distribution (NaN = draw or equal ratings):
higher_rated_wins
1.0    2620069
0.0    2099939
NaN     260951
```

Figure 5: Engineered Feature Summaries

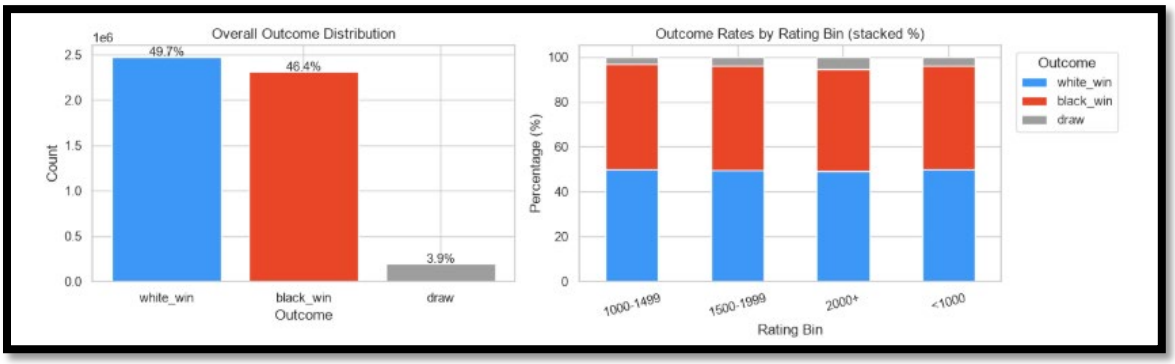
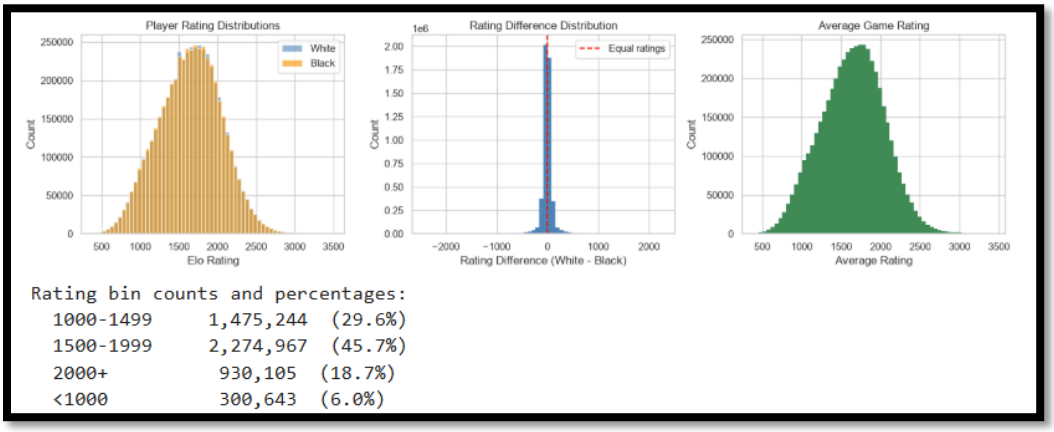


Figure 6: Outcome Distribution




```
Rating bin counts and percentages:
  1000-1499     1,475,244  (29.6%)
  1500-1999     2,274,967  (45.7%)
  2000+           930,105  (18.7%)
  <1000           300,643  (6.0%)
```

Figure 7: Player Rating Analysis and Rating Difference Analysis

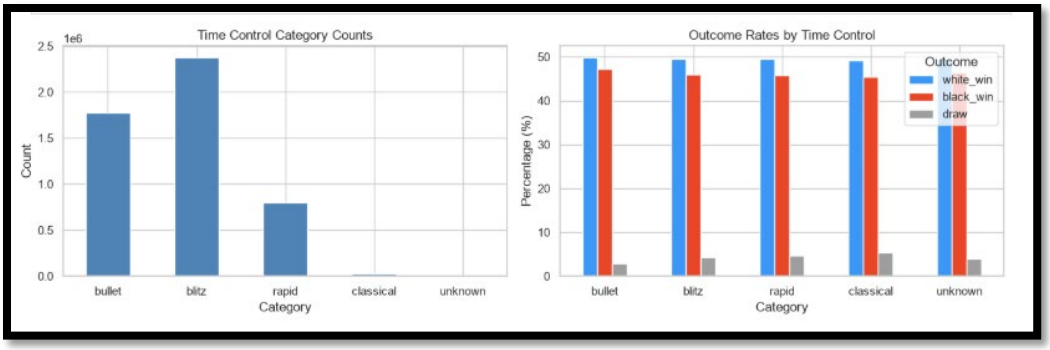


Figure 8: Game Length and Time Control Analysis

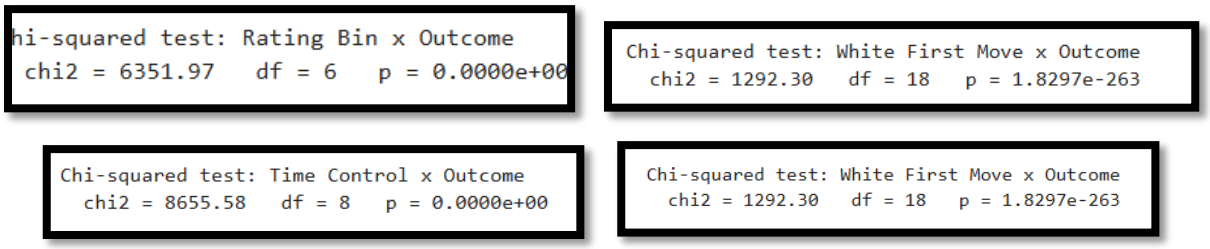
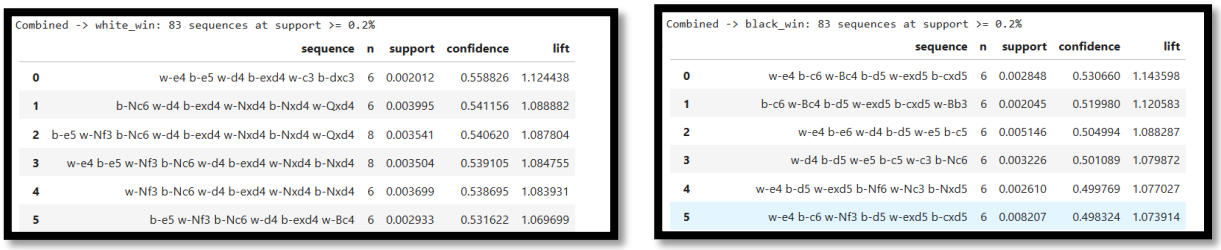


Figure 9: Chi-squared tests



Combined -> white_win: 83 sequences at support >= 0.2%

	sequence	n	support	confidence	lift
0	w-e4 b-e5 w-d4 b-exd4 w-c3 b-dxc3	6	0.002012	0.558826	1.124438
1	b-Nc6 w-d4 b-exd4 w-Nxd4 b-Nxd4 w-Qxd4	6	0.003995	0.541156	1.088882
2	b-e5 w-Nf3 b-Nc6 w-d4 b-exd4 w-Nxd4 b-Nxd4 w-Qxd4	8	0.003541	0.540620	1.087804
3	w-e4 b-e5 w-Nf3 b-Nc6 w-d4 b-exd4 w-Nxd4 b-Nxd4	8	0.003504	0.539105	1.084755
4	w-Nf3 b-Nc6 w-d4 b-exd4 w-Nxd4 b-Nxd4	6	0.003699	0.538695	1.083931
5	b-e5 w-Nf3 b-Nc6 w-d4 b-exd4 w-Bc4	6	0.002933	0.531622	1.069699

Combined -> black_win: 83 sequences at support >= 0.2%

	sequence	n	support	confidence	lift
0	w-e4 b-c6 w-Bc4 b-d5 w-exd5 b-cxd5	6	0.002848	0.530660	1.143598
1	b-c6 w-Bc4 b-d5 w-exd5 b-cxd5 w-Bb3	6	0.002045	0.519980	1.120583
2	w-e4 b-e6 w-d4 b-d5 w-e5 b-c5	6	0.005146	0.504994	1.088287
3	w-d4 b-d5 w-e5 b-c5 w-c3 b-Nc6	6	0.003226	0.501089	1.079872
4	w-e4 b-d5 w-exd5 b-Nf6 w-Nc3 b-Nxd5	6	0.002610	0.499769	1.077027
5	w-e4 b-c6 w-Nf3 b-d5 w-exd5 b-cxd5	6	0.008207	0.498324	1.073914

Figure 10: Top White and Black win Move Sequences

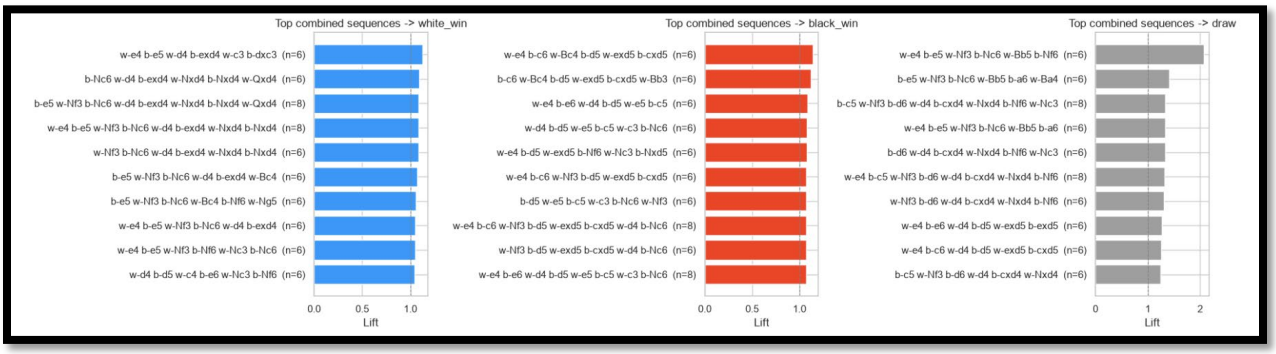


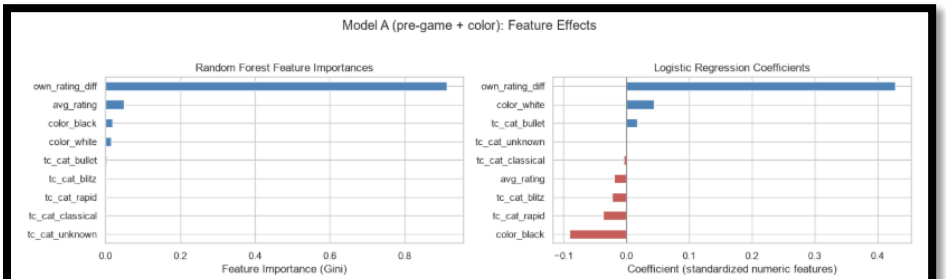
Figure 11: Top Combined Move Sequences by Outcome

	rating_bin	outcome	pattern	n	support	confidence	lift
0	<1000	white_win	w-e4 b-e5 w-Nf3 b-Nf6 w-Nxe5 b-Nxe4	6	0.005222	0.600000	1.203198
1	<1000	white_win	b-e5 w-Nf3 b-Nc6 w-Bc4 b-Nf6 w-Ng5	6	0.006230	0.584090	1.171292
2	<1000	white_win	w-e4 b-e5 w-Nf3 b-Nf6 w-Nc3 b-Nc6	6	0.005309	0.572055	1.147159
3	<1000	black_win	w-e4 b-e5 w-Qh5 b-Nf6 w-Qxe5+ b-Be7	6	0.005309	0.521303	1.132022
4	<1000	black_win	w-e4 b-e5 w-Qh5 b-Nc6 w-Bc4 b-g6	6	0.005671	0.516129	1.120786
5	<1000	black_win	w-e4 b-e5 w-Nf3 b-Nc6 w-Nc3 b-Bc5	6	0.005681	0.497073	1.079405
6	1000-1499	white_win	b-e5 w-Nf3 b-Nc6 w-d4 b-exd4 w-Nxd4 b-Nxd4 w-Qxd4	8	0.005910	0.541232	1.083183
7	1000-1499	white_win	w-e4 b-e5 w-Nf3 b-Nc6 w-d4 b-exd4 w-Nxd4 b-Nxd4	8	0.005869	0.539617	1.079950
8	1000-1499	white_win	b-Nc6 w-d4 b-exd4 w-Nxd4 b-Nxd4 w-Qxd4	6	0.006567	0.538501	1.077718
9	1000-1499	black_win	w-e4 b-c6 w-Nf3 b-d5 w-exd5 b-cxd5	6	0.008375	0.512424	1.094982
10	1000-1499	black_win	b-c6 w-Nf3 b-d5 w-exd5 b-cxd5 w-d4	6	0.005489	0.507595	1.084664

Figure 12: Move Sequences by Player Rating

```
Model A -- pre-game features only (target: own_win)
                                Accuracy  Precision  Recall      F1  ROC-AUC
Model
Majority Class                    0.5195     0.0000  0.0000  0.0000   0.5000
Higher-Rated Wins                 0.5529     0.5339  0.5478  0.5407      NaN
Logistic Regression (pre-game)    0.5586     0.5580  0.3911  0.4599   0.5867
Random Forest (pre-game)          0.5588     0.5560  0.4060  0.4693   0.5872
```

Figure 13: Model A Performance




```
]:  own_rating_diff     0.912216
    avg_rating          0.049935
    color_black         0.017886
    color_white         0.014941
    tc_cat_bullet       0.002619
    tc_cat_blitz        0.001134
    tc_cat_rapid        0.000764
    tc_cat_classical    0.000314
    tc_cat_unknown      0.000191
```

Figure 14: Model A Feature Effects

Model C -- pre-game features + checkpoint_ply + pattern-timing flags (target: own_win)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Majority Class	0.5323	0.0000	0.0000	0.0000	NaN
Higher-Rated Wins	0.5507	0.5187	0.5467	0.5323	NaN
Logistic Regression (checkpoint panel)	0.5582	0.5357	0.4164	0.4686	0.5809
Random Forest (checkpoint panel)	0.5496	0.6968	0.0655	0.1197	0.5822

Figure 15: Model C Performance

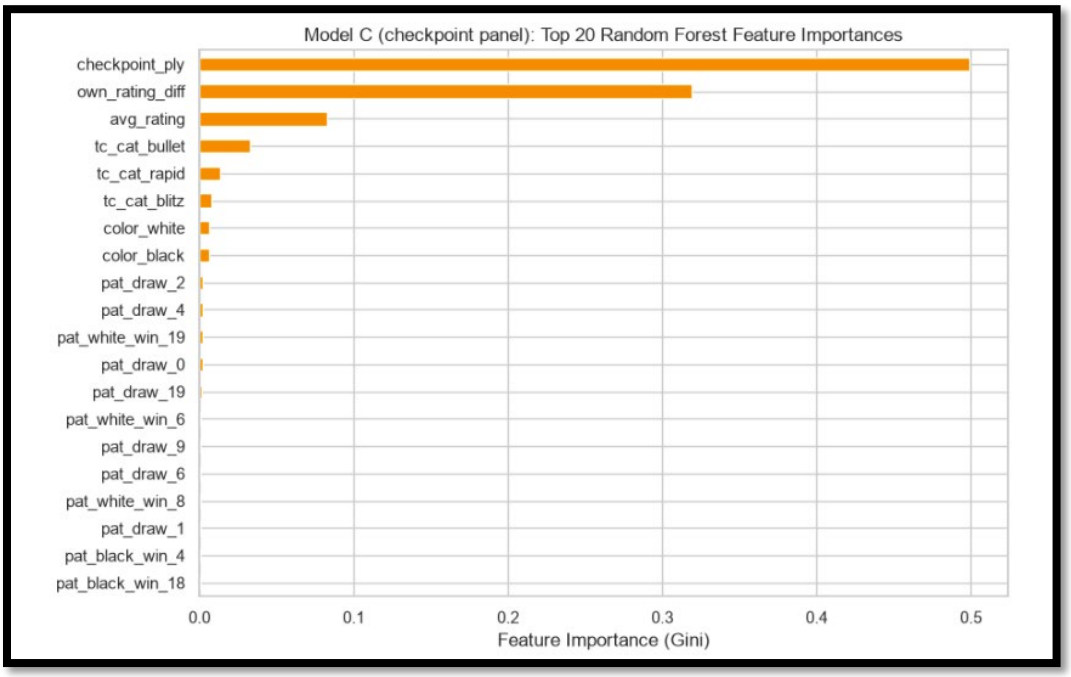


Figure 16: Model C Top 20 Random Forest Feature Importances

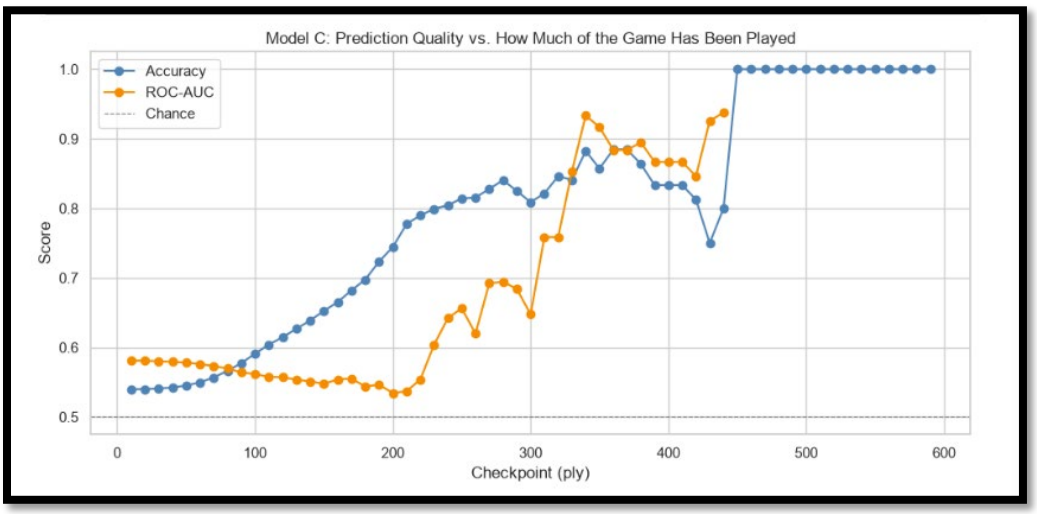


Figure 17: Model C Prediction Quality by Checkpoint

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest (pre-game)	0.558808	0.556020	0.406028	0.469332	0.587242
Random Forest (checkpoint panel)	0.549572	0.696846	0.065490	0.119728	0.582237

Figure 18: Model A vs Model C Performance Statistics

References

Chowdhary, Sandeep, Iacopo Iacopini, and Federico Battiston. "Quantifying Human Performance in Chess." *Scientific Reports*, vol. 13, 2023, article no. 2113, https://doi.org/10.1038/s41598-023-29369-7.

De Marzo, Giordano, and Vito D. P. Servedio. "Quantifying the Complexity and Similarity of Chess Openings Using Online Chess Community Data." *Scientific Reports*, vol. 13, 2023, article no. 5327, https://doi.org/10.1038/s41598-023-31658-w.

Lichess. "Lichess Database: Standard Rated Games, February 2026." *Lichess*, 2026, https://database.lichess.org/.

Samara, Khalid. "Machine Learning Approaches for Classifying Chess Game Outcomes: A Comparative Analysis of Player Ratings and Game Dynamics." *Electronics*, vol. 15, no. 1, 2026, article no. 1, https://doi.org/10.3390/electronics15010001.

Full Project Details

GitHub Repository: https://github.com/sgillihan/CSPB4502_DataMiningProject.git

Website: https://sgillihan.github.io/projects/data_mining-chess/